PolicyBench

Benchmarking no-tool tax-and-benefit estimation in frontier language models

Max Ghenis

Table of contents

Source: Article Notebook

1 Abstract

PolicyBench evaluates whether frontier language models can estimate household tax and benefit outputs from household facts without tools. This release covers the United States, drawing sampled households from the certified PolicyEngine populace microdataset, scored through policyengine.py 4.16.1. It evaluates 13 frontier models on 100 households across 18 output groups, and reports three complementary scores: an exact-match rate as the headline deployability bar (a prediction counts only if it matches the PolicyEngine reference to the dollar for amounts or to the eligibility flag for booleans), a within-1% hit rate as a near-miss-tolerant companion, and a continuous bounded score that awards partial credit for close answers. The headline is exact match because the leaderboard is household-impact-weighted: a naive unweighted exact rate would be dominated by the 84% of US reference outputs that are exact zeros — a hedge-to-zero model scores about 84% unweighted — but the household-impact weighting down-weights those zero-reference outputs, so the weighted exact rate is not compressed near the zero share and discriminates between models about as well as within-1%. On the weighted leaderboard the top model leads an always-zero baseline of 66.8% exact by 13.4 points, a margin comparable to within-1%, and exact spreads across the roster about as widely (the within-1% companion adds near-miss tolerance on top). The manuscript snapshot is a 100-household public preview, not a protected held-out leaderboard: the current public scenario explorer exposes prompts and reference outputs, so open-set leakage is a central limitation of any public ranking. In the frozen snapshot used for this manuscript (2026-06-14), the top-scoring US model on exact match is GPT-5.5 at 80.3, while GPT-5.4 nano is lowest at 62.2. Notably, the newest flagship is not the strongest on this task: Claude Opus 4.8 scores 72.5% exact, below the 77.3% of the prior-generation Claude Opus 4.7. Multi-step tax quantities and positive-dollar benefit cases are materially harder than zero cases. An exhaustive developer audit of every wrong cell traced all 3,300 scored misses to model errors, with zero PolicyEngine reference defects. The live benchmark is available at https://policybench.org.

2 Introduction

Household tax-and-benefit estimation sits between arithmetic and policy discussion. Each case has numeric labels, but generating those labels requires filing status, household composition, income concepts, program thresholds, and jurisdiction-specific rules. PolicyBench measures whether models can map household records to those outputs.

Most current language-model evaluations do not test this mapping directly. General math benchmarks emphasize symbolic manipulation and exact final answers (Cobbe et al. 2021; Hendrycks et al. 2021), while generic question-answering benchmarks emphasize recall or instruction following. PolicyBench instead asks whether models can transform household facts into policy outputs without access to tools or simulators.

This matters for public-facing calculators, analyst workflows, and tax-preparation or screening systems. The benchmark is intended to separate verbal policy fluency from household-level quantitative prediction.

3 Related work

The most relevant prior work is tax and statutory reasoning. SARA frames statutory interpretation, including tax-relevant rule application, as a language-understanding problem (Holzenberger et al. 2021). LegalBench broadens this view and includes tax-oriented numeric tasks such as sara_numeric (Guha et al. 2023). RuleArena evaluates rule-guided reasoning in regulation-style settings (Zhou et al. 2025), and TaxCalcBench evaluates frontier models on tax calculation from structured return-like inputs (Bock et al. 2025). Shanahan et al. (2025) report a similar split on the Volunteer Income Tax Assistance (VITA) test: models do better on tax knowledge questions than on open-ended calculation tasks.

PolicyBench also sits within a broader literature on quantitative reasoning benchmarks. Canonical math evaluations such as GSM8K and MATH normalized exact final-answer scoring for numeric tasks (Cobbe et al. 2021; Hendrycks et al. 2021). Recent work has questioned pure exact-match scoring in numeric QA and temporal reasoning, arguing for error-aware metrics instead (Abbood et al. 2025). PolicyBench treats this as a measurement design choice rather than a forced pick: it leads with the exact-match rate as the headline deployability bar (a tax filer or benefits caseworker can't ship "close"), retains a within-1% hit rate as a near-miss-tolerant companion, and reports a bounded continuous score that awards relative-error partial credit as a secondary tracking diagnostic. The benchmark panel is zero-inflated — 84% of reference outputs are exact zeros — so a naive unweighted exact rate would be dominated by zero cases a hedging model can predict; but the public leaderboard is household-impact-weighted, which down-weights those zero-reference outputs, so the weighted exact rate is not compressed near the zero share and ranks models about as sharply as within-1% while keeping the to-the-dollar deployability meaning.

Finance-domain benchmarks provide another relevant precedent. FinQA and TAT-QA both show that realistic quantitative reasoning often requires combining structured numbers with domain knowledge rather than solving stylized school-math problems (Chen et al. 2021; Zhu et al. 2021). PolicyBench differs in that its reference outputs are generated by executable tax-benefit microsimulation rather than annotated document questions, but the underlying motivation is similar: domain-specific quantitative reasoning deserves dedicated evaluation.

The nearest public-benefits benchmark is the concurrent Public Benefits Bench, built by Vals AI with the Center for Civic Futures and Code for America (Vals AI 2026). It evaluates models on 459 expert-validated SNAP guidance scenarios (230 in the published test set) spanning all 50 US states plus Guam and the Virgin Islands, and grades free-text answers against expert-written rubric criteria using a judge model that agreed with SNAP policy experts on 80.6% of grading decisions. Where PolicyBench isolates parametric knowledge by withholding tools, Public Benefits Bench varies tool access directly: across models, multi-turn conversation raises pass rates by roughly 7.6 percentage points and web search by about 6.9. The two benchmarks probe complementary failure surfaces — conversational guidance quality for a single program with tools available, versus no-tool numeric estimation across tax and transfer outputs scored deterministically against microsimulation references — and reach consistent conclusions. The best model passes 71.7% of rubric criteria under Public Benefits Bench's strongest condition, and its authors conclude that "no general-purpose AI model performs well enough to be trusted with SNAP benefits guidance," matching the deployability gaps PolicyBench reports for unaided calculation.

Finally, PolicyBench depends on two infrastructure literatures that are not themselves large language model (LLM) benchmarks. One is structured-output reliability, since the benchmark relies on multi-output JSON responses and parse coverage rather than free-form prose (Shorten et al. 2024). The other is tax-benefit microsimulation, where systems such as EUROMOD provide the methodological precedent for evaluating policy rules over household microdata (Sutherland and Figari 2013). There is also operational work on applying and evaluating AI systems in public-benefits settings, including caseworker-assist and Supplemental Nutrition Assistance Program (SNAP)-focused evaluations (Nava Labs 2026, 2025; ZenML LLMOps Database 2025). PolicyBench combines these strands into a public cross-model benchmark over household-level tax and benefit outputs.

4 Benchmark design

PolicyBench asks models to predict all benchmark outputs for a household in a single structured response. One response per household reduces repeated prompt cost and keeps the task at the household-return level rather than turning it into a sequence of unrelated one-output calls.

4.1 Headline metric: exact match

The headline metric is the exact-match rate: the share of requested outputs the model gets right to the dollar (or, for eligibility flags, to the boolean). This is the deployability bar. A tax filer, a benefit estimator, or a caseworker cannot ship "close" — a prediction off by $50 is no more usable than one off by $500, because neither matches the bottom line a downstream system or a household needs. For currency amounts, "exact" means within 1 currency unit of the reference value after numeric parsing. For binary outputs, “exact” means the parsed value is exactly the same 0/1 flag as the reference.

Exact match can be a deceptive headline in a zero-inflated panel — but only when the rate is computed unweighted. In the frozen manuscript snapshot, 84% of reference outputs are exact zeros: most sampled households are not eligible for any given program, owe no capital gains tax, do not phase into a refundable credit, and so on. A model that hedges to zero on every requested output earns full exact credit on every zero case, so a naive unweighted exact rate is dominated by those zeros — a hedge-to-zero model scores roughly 84% unweighted, leaving little room above the floor to separate models that actually understand the rules.

The public leaderboard corrects for this by household-impact weighting. Each requested output’s weight is its share of the household’s stake — |ref| / max(|household_net_income|, Σ |ref|) — averaged with calibrated household weights over the full populace population (the weighting construction is detailed under “Impact weighting” below). Because zero-reference outputs contribute zero weight, the weighting strips out exactly the rows a hedge-to-zero model gets for free. The weighted exact rate is therefore not compressed near the unweighted zero share: an always-zero baseline scores 66.8% weighted exact, well below the 84% unweighted floor, and the top model clears that baseline by 13.4 points — a margin comparable to within-1%. Across the 13-model roster, the weighted exact rate spreads about as widely as within-1% and ranks models in nearly the same order, so exact match discriminates between rule comprehension and zero-hedging about as well as within-1% while preserving the to-the-dollar deployability meaning.

The exact rate is aggregated to a household score and then to a country score, with the same population-derived output weights that the within-1% companion and the bounded score use. Equal household weight preserves comparability across scenarios and slices. The leaderboard reports US results on this fixed test set.

4.2 Near-miss companion: within 1%

The within-1% hit rate is the near-miss-tolerant companion to the headline: the share of requested outputs the model gets within one percent of the PolicyEngine reference value. For currency amounts, “within 1%” means |pred − ref| 0.01 × |ref| when the reference is nonzero and |pred| 1 (the same one-currency-unit absolute tolerance used by exact match) when the reference is zero. Binary outputs are requested as integer 0/1 eligibility flags and scored identically to the exact-match metric.

Within-1% relaxes the exact bar by tolerating rounding and small rate or parameter drift on positive-reference cases, so it credits a model that is consistently close but rarely to-the-dollar. It uses the same per-output household-impact weighting and the same household-equal aggregation as the headline exact rate, and it is reported alongside exact in every leaderboard table. Because both metrics share the weighting, the two rank models almost identically;

within-1% sits a few points above exact and is the right view when near-miss accuracy, rather than strict deployability, is the question.

4.3 Secondary metric: bounded continuous score

PolicyBench also reports a bounded 0-100 score that awards smooth partial credit for close-but-not-exact amount answers. This is informative for tracking conceptual progress year over year and captures the qualitative gap between a model that is consistently close and one that is consistently far from the reference value. For amount variables, the row-level bounded score is max(0, 1 - |pred - ref| / |ref|) when the reference value is nonzero and 1{pred = 0} when the reference value is zero. For binary outputs, it uses exact 0/1 matching. The exact, within-1%, within-5%, and within-10% hit rates are reported separately as threshold diagnostics; they are not averaged to create the bounded score.

PolicyBench also tracks mean absolute error and related error metrics. These remain secondary diagnostics under the headline exact-match rate, the within-1% near-miss companion, and the bounded score. The three-metric design follows recent numeric-evaluation literature that questions pure exact-match scoring (Abbood et al. 2025) while keeping exactness as the production bar.

All requested US outputs are annual amounts or annual eligibility indicators for tax year 2026. For currency amounts, the "exact" hit rate means within 1 currency unit of the reference value after numeric parsing. Percentage-threshold hit rates use relative error when the reference value is nonzero and the same 1 currency-unit absolute tolerance when the reference value is zero. Binary outputs are parsed as numeric 0/1 eligibility flags and scored by exact classification accuracy. Missing, unparseable, or non-0/1 answers receive zero score for that requested output.

Aggregation proceeds in three steps. First, each household-output prediction receives a 0-100 score. Second, each household receives one model score: requested output rows are weighted by output-group weights constructed from the full weighting population, then renormalized within the household over outputs that are actually requested for that household. Person-level coverage flags are scored at the person row; their output-group weight is split across the relevant people in the household and is based on PolicyEngine value proxies where available. Third, the country score averages those household scores with equal household weight. The leaderboard reports US results on this fixed test set. Equal-output-group scores and other alternative views are reported as sensitivity checks.

The benchmark requires each model response to include numeric answers and one explanation per requested output. Both the exact-match rate and the bounded score use only the numeric answers. Explanations are retained for scenario exploration and qualitative error analysis; they should not be interpreted as faithful traces of model reasoning.

Because explanations are required, the canonical task measures policy estimation under a public-facing structured-response contract, not isolated arithmetic accuracy. Prompt fairness is part of the benchmark contract. The current release uses one prompt template per country, with no model-specific tuning. Models receive the same household facts, the same requested outputs, and no web or tool access. The prompt states that unlisted numeric inputs are 0, unlisted boolean or status facts are false, and household characteristics are constant over the tax-benefit year. Provider-specific differences are limited to the response transport needed to obtain structured output and the reasoning configuration documented per model in Table 2: models whose provider enables reasoning by default run with extended thinking against a fixed shared completion budget, one model runs with an explicit reasoning-effort setting, and every other model receives no reasoning-control parameters. Reasoning configuration is therefore not uniform across the roster. Sampling parameters are never set; temperature and related decoding controls stay at provider defaults for every model. A rerun at strictly unconfigured provider defaults for all models is planned as a sensitivity release.

4.4 Frozen snapshot and open-set status

Leaderboard positions are version-sensitive, so manuscript claims refer to the frozen source-run exports in Table 1 rather than to the live site. The committed source-run exports are the manuscript artifacts. The public site exposes the current prompts, predictions, explanations, and reference outputs for transparency. This makes the public leaderboard open-set: models, model providers, or benchmark users could learn from the released cases before later runs. Protected leaderboard claims would require a separate held-out or rotating set.

Source: Article Notebook

Table 1: Frozen manuscript snapshot.

	Item	Value
0	Snapshot date	2026-06-14
1	Model response date	2026-06-12 to 2026-06-13
2	Policy period	US tax year 2026
3	Frozen export	paper/snapshot/20260501/runs/us_full_run_20260612_policye
4	Frozen export SHA-256 prefix	US 7d9fc182ad6e
5	Snapshot manifest	paper/snapshot/20260501/manifest.json
6	Audit annotations	paper/snapshot/20260501/annotations
7	US scenarios SHA-256 prefix	71b16212f0c0
8	US reference outputs SHA-256 prefix	59860ff2d3c2
9	Benchmark spec SHA-256 prefix	ce233f8cbb05
10	Frozen prompt payload SHA-256 prefix	US 1c3f94362d0b
11	Scoring code SHA-256 prefix	9d41fa97849a
12	US run label	us_full_run_20260612_policyengine_4_16_1_populace

Table 1: Frozen manuscript snapshot.

	Item	Value
13	Household sample seed	42
14	PolicyEngine.py	4.16.1
15	PolicyEngine-US	policyengine-us 1.723.0
16	US dataset	populace_us_2024 (populace-us-2024-5da5a95-20260611)
17	Households	100 US
18	Models	13
19	Output groups	18 US
20	Condition	No tools, no web access, one structured response per household
21	Response contract	Numeric answer and non-empty explanation for every requested

Source: Article Notebook

	Model	PolicyBench ID	Provider ID	Pars
0	Claude Haiku 4.5	claude-haiku-4.5	claude-haiku-4-5-20251001	1,98
1	Claude Opus 4.7	claude-opus-4.7	claude-opus-4-7	1,98
2	Claude Opus 4.8	claude-opus-4.8	claude-opus-4-8	1,98
3	Claude Sonnet 4.6	claude-sonnet-4.6	claude-sonnet-4-6	1,98
4	Gemini 3 Flash Preview	gemini-3-flash-preview	gemini/gemini-3-flash-preview	1,98
5	Gemini 3.1 Flash Lite Preview	gemini-3.1-flash-lite-preview	gemini/gemini-3.1-flash-lite-preview	1,98
6	Gemini 3.1 Pro Preview	gemini-3.1-pro-preview	gemini/gemini-3.1-pro-preview	1,98
7	Gemini 3.5 Flash	gemini-3.5-flash	gemini/gemini-3.5-flash	1,98
8	GPT-5.4 mini	gpt-5.4-mini	gpt-5.4-mini	1,98
9	GPT-5.4 nano	gpt-5.4-nano	gpt-5.4-nano	1,98
10	GPT-5.5	gpt-5.5	gpt-5.5	1,98
11	Grok 4.3	grok-4.3	xai/grok-4.3	1,98
12	Grok Build 0.1	grok-build-0.1	xai/grok-build-0.1	1,98

Source: Article Notebook

5 Data and scenario construction

5.1 United States

The US benchmark is built from the certified PolicyEngine populace dataset (populace_us_2024) using PolicyEngine US. The dataset contains 160,858 people in 75,112 households, with household, tax-unit, SPM-unit, family, marital-unit, and person records for PolicyEngine calculations. The sampled households are filtered to keep a single-tax-unit, single-family, single-Supplemental Poverty Measure (SPM)-unit structure with at least one adult and a supported filing status. Of the 75,112 households, 63,128 (84.0%) pass the filter and form the eligible draw. The 16.0% excluded by the filter include multi-tax-unit households (e.g., adult roommates), multi-family households, multi-SPM-unit households, and households whose head reports a filing status outside the supported set. These excluded compositions are exactly the kind of cases where federal/state credit allocations and benefit-unit rules become hardest, so the eligible draw is a tractable subset rather than the full distribution of US households. Prompts include nonzero promptable raw inputs across relevant entities rather than a hand-curated summary, so the models see many of the same facts the simulator receives. Filing status is not stated in the prompt; the reference computation infers it from tax-unit role flags. Models therefore see the same household facts that drive the reference filing-status assignment, but they do not receive that assignment as a label.

The current US release requests 18 scored output groups spanning federal income tax, refundable credits, payroll and self-employment tax, state and local income tax, Supplemental Nutrition Assistance Program (SNAP), Supplemental Security Income (SSI), Temporary Assistance for Needy Families (TANF), school-meal eligibility, and person-level coverage eligibility for the Special Supplemental Nutrition Program for Women, Infants, and Children (WIC), Medicaid, the Children's Health Insurance Program (CHIP), Medicare, Head Start, and Early Head Start. The source run also requested the ACA Premium Tax Credit (PTC), but explanation audits showed that the prompt could be misleading when households lacked plan-specific Marketplace information. We therefore preserve those raw responses but exclude PTC from the canonical scored leaderboard until the prompt contract is revised.

The output scope is intentionally narrower than the full PolicyEngine model. Table 3 summarizes the inclusion rule. The benchmark asks for WIC eligibility rather than a WIC dollar amount; WIC dollar values are used only as impact-weight proxies for coverage flags, not as requested model outputs.

Source: Article Notebook

Ta

	Scope decision	Rationale
0	Included	Direct tax, credit, benefit, health-support, and coverage outputs that a househol
1	Excluded	Intermediate tax bases, payroll subcomponents, and outputs that mainly require
2	Binary coverage outputs	Requested as 0/1 eligibility flags and scored as classification tasks; their dollar v
3	WIC	The benchmark asks for person-level WIC eligibility. It does not ask models to e

Source: Article Notebook

5.2 Reference-output credibility

PolicyBench treats PolicyEngine outputs as benchmark reference outputs, not as administrative records. The reference source is nevertheless stronger than an ad hoc answer key: PolicyEngine is open source, used for household calculators and reform analysis, and externally checked in specific domains. No. 10 Downing Street's data science team adapted PolicyEngine's open-source microsimulation model for experimental policy simulation, with validation against external projections before use (Woodruff 2026; Ghenis 2026). In the US, PolicyEngine (2024) reports matching the National Bureau of Economic Research (NBER) TAXSIM-35 model (Feenberg and Coutts 1993) to the cent on the vast majority of cases for the 2021 tax year across hundreds of thousands of tax units per state, with state-specific differences documented in the integration tests. We do not restate that comparison as a single percentage because the published source uses qualitative phrasing rather than a headline accuracy number. PolicyEngine has also signed a memorandum of understanding (Ghenis and Makarchuk 2025) with the Federal Reserve Bank of Atlanta for future validation work against its Policy Rules Database (2026). The Atlanta Fed sources are a caveat rather than evidence of completed validation for this benchmark: they document planned collaboration and the comparison source, not finished checks of the frozen PolicyBench outputs. Taken together, these sources support using PolicyEngine as a transparent reference implementation with partial external validation, but they do not validate every benchmark output.

This does not make PolicyEngine infallible. During benchmark development, we performed a manual, developer-led discrepancy review, with LLM assistance used to triage and summarize candidate cases surfaced by model explanations. Table 4 summarizes the main discrepancy classes reviewed before the frozen snapshot. In those reviewed classes, discrepancies reflected model mistakes, prompt ambiguity later corrected, or upstream data/model issues fixed before the frozen snapshot; the reviewed discrepancy classes did not identify unresolved PolicyEngine reference-output defects.

After freezing the snapshot and completing response-contract repairs, we annotated every wrong model-output row and every scenario-output case with at least one wrong row. Table 5 reports that all 3,300 rows receiving less than full score have row-level and case-level annotations. The final row-level source is llm_error; no frozen-snapshot wrong row remains classified as a prompt ambiguity, unresolved reference-model issue, unresolved reference-data issue, parse-contract failure, or needs-review item. The audit is still developer-led rather than independent external validation, but it is exhaustive over scored misses in the repaired frozen snapshot.

Source: Article Notebook

Table 4: Development discrepancy review

	Discrepancy class	Review outcome
0	US federal credits	Reviewed cases where explanations omitted EITC or refundable CTC; disc
1	US SNAP, SSI, and Medicaid	Reviewed false-zero and false-positive explanations; common model failure

Table 4: Development discrepancy review

	Discrepancy class	Review outcome
2	US payroll and overtime	Reviewed overtime-related cases during development; upstream data/mode
3	US state and local income tax	Reviewed state-rule and bracket misses; explanations applied wrong state

Source: Article Notebook

Table 5: Exhaustive annotation coverage for scored misses in the frozen snapshot

	Country	Wrong rows audited	LLM response errors	Unresolved prompt/reference rows	Missing row a
0	US	3300	3300	0	0
1	Total	3300	3300	0	0

Source: Article Notebook

6 Results

6.1 United States leaderboard

The US leaderboard for the frozen manuscript snapshot is shown in Table 6. The top three models in that snapshot are GPT-5.5 (80.3 exact, 95% CI 77.0–83.3; 82.6 within-1%), Gemini 3.1 Pro Preview (77.8 exact, 95% CI 74.4–81.2; 78.3 within-1%), and Claude Opus 4.7 (77.3 exact, 95% CI 74.3–80.3; 78.3 within-1%).

Source: Article Notebook

Table 6: Top US benchmark models in the frozen manuscript snapshot, ranked by the headli exact-match rate with the within-1% near-miss companion alongside. Exact 95% C and rank ranges are household-resampling bootstrap intervals (see Uncertainty).

	Model	Exact %	Exact 95% CI	Exact rank	Within 1% %	Within 10%	Bounded
0	GPT-5.5	80.3	77.0–83.3	1–2	82.6	88.3	92.0
2	Gemini 3.1 Pro Preview	77.8	74.4–81.2	1–6	78.3	83.2	91.1
3	Claude Opus 4.7	77.3	74.3–80.3	2–9	78.3	83.5	88.6
1	Grok 4.3	77.2	74.1–80.2	2–9	79.1	83.3	87.1
4	Claude Sonnet 4.6	77.0	73.8–80.2	2–9	78.2	86.2	89.8

Source: Article Notebook

6.2 Uncertainty

Each leaderboard score is a mean over a sample of 100 households, so the rankings carry sampling uncertainty. We quantify it with a household-level bootstrap: the 100 households are resampled with replacement 10,000 times, and each model's population-weighted exact-match rate is recomputed on every resample. The resampling unit is the household, carried with its full output vector, so within-household correlations across outputs are preserved. The same resampled set is used for every model in each replicate (a paired bootstrap), so the rank ranges in Table 6 reflect genuine ordering uncertainty rather than independent per-model noise.

Sampling uncertainty is substantial at this sample size. In the frozen snapshot, 8 of the other 12 models have an exact-match 95% interval that overlaps the leader's. Small differences in the leaderboard ordering should therefore be read as ties rather than as a strict ranking, and the rank-range column makes the set of plausible positions explicit. These intervals are a manuscript artifact: the public leaderboard and app report point estimates only, because the benchmark sample, not run-to-run model variation, is the dominant source of uncertainty here.

6.3 Cost and latency

Accuracy is one axis; the price and speed of a no-tools answer are another. Table 7 reports each model's estimated cost to compute one household's full output vector and its median per-household latency, next to the headline exact rate. Per-household cost spans \\\\$0.002 (GPT-5.4 nano) to \\\\$0.293 (Claude Opus 4.7), and median per-household latency 3 s (Gemini 3.1 Flash Lite Preview) to 135 s (Grok Build 0.1). Neither dimension cleanly tracks accuracy: the top-scoring model, GPT-5.5 at 80.3% exact, sits mid-pack at \\\\$0.119 per household and 66 s.

Source: Article Notebook

Table 7: Estimated cost and median latency per household for the frozen US snapshot, alongside the headline exact-match rate. Cost is reconstructed from logged token usage at provider list prices; latency is the median across households of each household's summed request-time.

	Model	Exact (%)	Cost / household	Latency (median)
0	GPT-5.5	80.3	\$0.119	66 s
1	Gemini 3.1 Pro Preview	77.8	\$0.085	45 s
2	Claude Opus 4.7	77.3	\$0.293	53 s
3	Grok 4.3	77.2	\$0.015	22 s
4	Claude Sonnet 4.6	77.0	\$0.186	114 s
5	Gemini 3 Flash Preview	76.8	\$0.048	58 s
6	Gemini 3.5 Flash	76.1	\$0.056	24 s

Table 7: Estimated cost and median latency per household for the frozen US snapshot, alongside the headline exact-match rate. Cost is reconstructed from logged token usage at provider list prices; latency is the median across households of each household's summed request-time.

	Model	Exact (%)	Cost / household	Latency (median)
7	Grok Build 0.1	76.0	\$0.045	135 s
8	Gemini 3.1 Flash Lite Preview	76.0	\$0.002	3 s
9	Claude Opus 4.8	72.5	\$0.255	55 s
10	Claude Haiku 4.5	71.7	\$0.055	48 s
11	GPT-5.4 mini	70.4	\$0.006	5 s
12	GPT-5.4 nano	62.2	\$0.002	6 s

Source: Article Notebook

Cost is reconstructed from each call's logged prompt and completion tokens at provider list prices through litellm; for grok-build-0.1, which litellm's price map did not yet cover, we apply the provider's published rate of $1 and $2 per million input and output tokens. Latency is wall-clock request time as observed by the harness, summed within each household and taken as the median across the 100 households. The per-call timer spans any provider-side retry or rate-limit backoff, so the median is reported in preference to the mean, which a few throttled calls inflate. Both are properties of this run's providers, prompts, and conditions at evaluation time rather than fixed model constants — an agent that handed the calculation to a deterministic engine would move this frontier rather than sit on it.

6.4 Simple baselines

Simple baselines help interpret score levels in a zero-heavy benchmark. Table 8 reports an always-zero response and a median-reference-by-output response on the same frozen household sample, scored with the household-impact weighting. These are not model competitors; they show how much of each metric can be earned without household-specific policy calculation. The always-zero baseline scores 66.8% on the headline exact-match rate — well below the 84% of outputs that are exact zeros, because the impact weighting down-weights the zero-reference rows the hedge gets for free. That is why exact match is not a trivial bar above the baseline: the top model clears it by 13.4 points, and the within-1% companion shows the same separation with near-miss tolerance.

Source: Article Notebook

Table 8: Simple baseline scores on the frozen manuscript snapshot, household-impact-weighted. The always-zero exact rate sits well below the unweighted zero share because the weighting down-weights zero-reference outputs.

	Baseline	Exact %	Within 1% %	Bounded score
0	Always zero	66.8	66.8	66.8
1	Median reference by output	61.0	61.0	64.2

Source: Article Notebook

6.5 Sensitivity to benchmark view

The primary leaderboard gives each household equal weight. Table 9 reports several alternative views, including the older equal-output-group score. The top model is stable across several views, but lower ranks move across sensitivity views. Scores are therefore best read with the weighting choice visible rather than collapsed into a single ordering.

Source: Article Notebook

Table 9: Leaderboard sensitivity to alternative scoring views.

	View	Rank 1	Rank 2	Rank 3
0	Exact-match headline	GPT-5.5 (80.3)	Gemini 3.1 Pro Preview (77.8)	Claude Op
1	Within-1% companion	GPT-5.5 (82.6)	Grok 4.3 (79.1)	Gemini 3.
2	Bounded score	GPT-5.5 (92.0)	Gemini 3.1 Pro Preview (91.1)	Gemini 3
3	Equal-output-group score	GPT-5.5 (95.5)	Gemini 3.1 Pro Preview (94.8)	Gemini 3
4	Amount outputs only	GPT-5.5 (94.5)	Claude Sonnet 4.6 (92.7)	Gemini 3
5	Positive reference cases only	Gemini 3.5 Flash (82.5)	GPT-5.5 (76.9)	Gemini 3
6	Zero reference cases only	Gemini 3.1 Pro Preview (98.1)	GPT-5.5 (97.8)	Grok 4.3 (

Source: Article Notebook

The bounded score is a global-output-weighted average of row scores. Amount outputs use $\max(0, 1 - |\text{pred} - \text{ref}| / |\text{ref}|)$ when the reference is nonzero and $1\{\text{pred} = 0\}$ when the reference is zero. Binary outputs use exact 0/1 matching. The exact-match rate uses the same weighting and aggregation but replaces the amount row score with a strict indicator: $1\{\text{pred} = \text{ref}\}$ to the dollar. Each output group's default household-impact weight is computed from a full weighting population, not from the 100 benchmark households: the certified PolicyEngine US populace dataset. For each source household, the share is |ref_ij| / max(|household_net_income_i|, Σ_k |ref_ik|); shares are averaged with calibrated household weights and renormalized so output weights sum to one. The max(...) denominator anchors per-household shares to net income when net income is the dominant flow and falls back to the gross tax-benefit flow only when programs cancel each other out, so a $1 benefit to a high-earner contributes essentially zero weight and per-household shares never exceed one. Booleans carry weight through PolicyEngine's paired per-capita value (for example medicaid_value), so eligibility calls are graded as accuracy but weighted by the dollar stake. Table 10 compares the bounded-score ranking against two opt-in alternatives shown for transparency: equal weighting (every output the same) and budget-weighted (each output's share of total absolute reference dollars in the same full weighting population).

Source: Article Notebook

Table 10: Top country ranks under the bounded secondary metric across three weightings (household, equal, budget).

	Weighting	Rank 1	Rank 2	Rank 3
0	Household-weighted (default)	GPT-5.5 (92.0)	Gemini 3.1 Pro Preview (91.1)	Gemini 3 Flash Preview
1	Equal weights	GPT-5.5 (95.3)	Gemini 3.1 Pro Preview (94.3)	Gemini 3 Flash Preview
2	Budget-weighted	GPT-5.5 (91.1)	Claude Sonnet 4.6 (89.0)	Claude Opus 4.7 (88.1)

Source: Article Notebook

6.6 Hardest benchmark targets

The lowest-scoring US variables are multi-step tax quantities and sparse positive-dollar benefits. As shown in Table 11, federal income tax before refundable credits, state income tax before refundable credits, state refundable credits, SNAP, and medicaid eligibility are the hardest US outputs by bounded score in the frozen manuscript snapshot.

Source: Article Notebook

Table 11: Hardest US variables by bounded score (the secondary tracking metric, used here because both exact-match and within-1% rates on the hardest variables compress to single-digit percentages and don't discriminate well).

	Variable	Score	Exact	Within 1%	Within 10%
0	federal_income_tax_before_refundable_credits	75.1	47.1	49.2	58.8
15	state_income_tax_before_refundable_credits	77.9	53.3	54.9	65.7
16	state_refundable_credits	82.2	78.8	79.0	79.6
13	snap	83.5	76.8	77.5	80.7
8	person_medicaid_eligible	87.8	87.8	87.8	87.8

Source: Article Notebook

6.7 Zero and positive cases

Overall hit rates can overstate performance on sparse programs because correct zeros are common. Table 12 therefore reports within-10% performance for all cases, positive-reference cases, and zero-reference cases. The gap is large for several outputs: models often identify that a program or tax does not apply, but miss the amount when the reference value is positive.

Source: Article Notebook

Table 12: US within-10% performance by zero versus positive reference cases.

	Variable	All cases	Positive cases	Zero cases
0	federal_income_tax_before_refundable_credits	58.8	25.6	89.5
1	state_income_tax_before_refundable_credits	65.7	31.8	92.3
2	state_refundable_credits	79.6	7.3	98.8
3	snap	80.7	19.2	96.1
4	payroll_tax	85.0	79.4	94.9
5	federal_refundable_credits	86.9	34.9	94.7
6	person_medicaid_eligible	87.8	75.5	91.6
7	person_head_start_eligible	91.5	—	91.5

Source: Article Notebook

7 Failure modes

The benchmark surfaces a few recurring failure patterns.

First, models miss positive tax and benefit quantities more often than zero cases. The lowest-scoring outputs by bounded score are federal income tax before refundable credits, state income tax before refundable credits, state refundable credits, SNAP, and medicaid eligibility. These require the model to choose the right income concepts, exclusions, program thresholds, and sequencing before applying any final subtraction. Positive benefit cases remain difficult, so the result should not be read as a general claim that benefits are easy.

Second, joint accuracy across interacting components is lower than marginal accuracy on either component. Table 13 shows within-10% accuracy for federal_refundable_credits, state_refundable_credits, and the conjunction of both within the same household. The joint hit rate is consistently lower than either marginal hit rate, so leaderboard scores that average across outputs understate how often a model gets a single household's federal/state credit allocation jointly correct.

Source: Article Notebook

Figure 1: Output-level performance on zero-reference and positive-reference cases.

Table 13: US within-10% accuracy on federal vs state refundable credits and the household-level joint.

	Model	Federal within 10%	State within 10%	Joint within 10%
10	GPT-5.5	96.0	83.0	82.0
4	Gemini 3 Flash Preview	92.0	82.0	78.0
6	Gemini 3.1 Pro Preview	90.0	82.0	76.0
7	Gemini 3.5 Flash	90.0	79.0	74.0
3	Claude Sonnet 4.6	93.0	79.0	73.0
5	Gemini 3.1 Flash Lite Preview	86.0	79.0	73.0
9	GPT-5.4 nano	86.0	79.0	72.0
12	Grok Build 0.1	88.0	78.0	72.0
2	Claude Opus 4.8	84.0	79.0	70.0
11	Grok 4.3	84.0	79.0	70.0
8	GPT-5.4 mini	81.0	79.0	69.0
1	Claude Opus 4.7	85.0	78.0	67.0
0	Claude Haiku 4.5	75.0	79.0	64.0

Source: Article Notebook

Third, structured-output reliability is part of the benchmark contract. Missing or unparseable numeric values are not dropped. Appendix A documents parser recovery, bounded full-response retries, and row-level repairs that brought the canonical manuscript snapshot to zero missing numeric values or explanations while preserving the failed attempts.

8 Limitations

PolicyBench is not a substitute for a production tax-and-benefit calculator. Several caveats matter:

- output-contract reliability required a repair workflow, and the raw failed attempts are preserved separately from the repaired canonical prediction files
- zero-heavy outputs require separate positive-case interpretation
- the choice of headline metric (exact match), near-miss companion (within-1%), and secondary tracking metric (bounded score) is one benchmark view; all three are reported, but downstream evaluations should also consider error-magnitude metrics (mean absolute error, mean absolute percentage error) where appropriate
- the public scenario explorer exposes the current test set and reference outputs, so open-set leakage is a prominent limitation rather than a minor implementation detail
- the 100-household manuscript snapshot should be treated as a preview until larger frozen runs are published

- the scored-miss audit is exhaustive over this frozen snapshot, but it is developer-led and not an independent validation of every reference value
- benchmark success should not be interpreted as policy-advice readiness

The current paper is therefore an evaluation of model performance under a specific structured-output benchmark, not a general certification of tax or benefit competence.

9 Conclusion

PolicyBench shows a consistent pattern across the US benchmark. Models often identify non-applicability, but positive tax and benefit amounts remain difficult, especially for multi-step income tax, payroll tax, and positive benefit cases. In the frozen manuscript snapshot, GPT-5.5 is the top-scoring US model.

These results support a narrow conclusion: unaided frontier models still struggle to reproduce selected household-level microsimulation outputs under a structured public benchmark. They do not show that models cannot assist policy analysis, and they do not validate PolicyEngine outputs as administrative truth. They suggest that future evaluations should separate no-tool estimation, tool-using system design, and reference-output validation more explicitly.

Next steps are to expand country coverage, increase frozen sample sizes, and add protected or rotating evaluation sets so that public rankings are less exposed to open-set leakage. The benchmark should also continue reporting sensitivity views, because scores are useful summaries only when their weighting choices are visible.

10 Appendix A: Structured-output audit

The benchmark requires one numeric value and one explanation for every requested output. Parse failures are benchmark failures, not missing data to drop from the denominator. Before publishing the manuscript snapshot, we audited missing numeric values and explanations, extended the parser only where explicit variable-keyed value and explanation fields were recoverable, and re-elicited the few remaining missing cells individually. Table 14 summarizes that sequence.

Source: Article Notebook

Table 14: Structured-output parser

	Step	Finding
0	Initial parse-contract audit	The source run was audited for rows missing a parsed numeric value or non-e
1	Parser repair	The parser was extended to recover explicit ‘value‘ and non-empty ‘explanatio

Table 14: Structured-output parser

	Step	Finding
2	Full-response retries	No full-response retries were required for the June populace run; whole-respo
3	Row-level repairs	Five missing cells were re-elicited individually against the same model, housel
4	Final parse coverage	The repaired manuscript snapshot has zero missing parsed numeric values and
5	Preservation rule	The snapshot retains the raw provider responses in the compressed source-ru

Source: Article Notebook

No full-response retry or bulk row-repair rounds were required for the June populace run; the five missing cells noted above were re-elicited individually against the same model, household, and output, and the resulting canonical prediction files have zero missing numeric values or explanations.

11 Competing interests

The author is affiliated with PolicyEngine, which develops the microsimulation software used to produce the benchmark reference outputs.

Source: Article Notebook

Abbood, Auss, Zaiqiao Meng, and Nigel Collier. 2025. "Time to Revisit Exact Match." Findings of the Association for Computational Linguistics: EMNLP 2025 (Suzhou, China), 11903–26. https://doi.org/10.18653/v1/2025.findings-emnlp.637.

Bock, Michael R., Kara Molisee, Zachary Ozer, and Sumit Shah. 2025. "TaxCalcBench: Evaluating Frontier Models on the Tax Calculation Task." arXiv Preprint arXiv:2507.16126, ahead of print. https://doi.org/10.48550/arXiv.2507.16126.

Chen, Zhiyu, Wenhu Chen, Charese Smiley, et al. 2021. "FinQA: A Dataset of Numerical Reasoning over Financial Data." arXiv Preprint arXiv:2109.00122. https://arxiv.org/abs/2109.00122.

Cobbe, Karl, Vineet Kosaraju, Mohammad Bavarian, et al. 2021. "Training Verifiers to Solve Math Word Problems." arXiv Preprint arXiv:2110.14168. https://arxiv.org/abs/2110.14168.

Federal Reserve Bank of Atlanta. 2026. Policy Rules Database. Federal Reserve Bank of Atlanta. https://www.atlantafed.org/what-we-study/workforce-development/advancing-careers-for-low-income-families/policy-rules-database.

Feenberg, Daniel R., and Elisabeth Coutts. 1993. "An Introduction to the TAXSIM Model." Journal of Policy Analysis and Management 12 (1): 189–94.

Ghenis, Max. 2026. PolicyEngine Powers Rapid Policy Analysis at No 10 Downing Street. PolicyEngine. https://www.policyengine.org/us/research/policyengine-10-downing-street.

Ghenis, Max, and Pavel Makarchuk. 2025. PolicyEngine and Atlanta Fed Sign Memorandum of Understanding for Policy Rules Database Validation. PolicyEngine. https://www.policyengine.org/us/research/policyengine-atlanta-fed-mou-prd.

Guha, Neel, Julian Nyarko, Daniel E. Ho, et al. 2023. "LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models." arXiv Preprint arXiv:2308.11462. https://arxiv.org/abs/2308.11462.

Hendrycks, Dan, Collin Burns, Saurav Kadavath, et al. 2021. "Measuring Mathematical Problem Solving with the MATH Dataset." NeurIPS Datasets and Benchmarks Track. https://arxiv.org/abs/2103.03874.

Holzenberger, Nils, Benjamin Van Durme, Sarah Lawsky, and Kyle Richardson. 2021. "Factoring Statutory Reasoning as Language Understanding Challenges." arXiv Preprint arXiv:2105.07903. https://arxiv.org/abs/2105.07903.

Nava Labs. 2025. Experimenting with AI-Powered Tools in Public Benefits. Case study. https://www.navapbc.com/case-studies/ai-tools-public-benefits.

Nava Labs. 2026. Evaluating a GenAI-Powered Assistive Chatbot for Caseworkers. Case study. https://www.navapbc.com/case-studies/evaluating-ai-assistive-chatbot-caseworkers.

PolicyEngine. 2024. PolicyEngine Launches State Income Tax Modeling Nationwide. PolicyEngine. https://www.policyengine.org/us/research/state-tax-model-beta.

Shanahan, Catherine, Emma McCarthy, Yan Zhao, et al. 2025. "Performance of LLMs on VITA Test: Potential for AI-Assisted Tax Returns for Low Income Taxpayers." Artificial Intelligence and Law, ahead of print. https://doi.org/10.1007/s10506-025-09465-7.

Shorten, Connor, Charles Pierse, Thomas Benjamin Smith, et al. 2024. "StructuredRAG: JSON Response Formatting with Large Language Models." arXiv Preprint arXiv:2408.11061, ahead of print. https://doi.org/10.48550/arXiv.2408.11061.

Sutherland, Holly, and Francesco Figari. 2013. "EUROMOD: The European Union Tax-Benefit Microsimulation Model." International Journal of Microsimulation 6 (1): 4–26. https://doi.org/10.34196/ijm.00075.

Vals AI. 2026. Public Benefits Bench. Benchmark report, in partnership with the Center for Civic Futures and Code for America. https://www.vals.ai/benchmarks/public-benefits-bench.

Woodruff, Nikhil. 2026. Informing Policy Using Micro-Simulations. No10 Innovation Fellowship. https://fellows.ai.gov.uk/articles/nikhil-woodruff-micro-simulation.

ZenML LLMOps Database. 2025. Building and Automating Comprehensive LLM Evaluation Framework for SNAP Benefits. Industry writeup. https://www.zenml.io/llmops-database/building-and-automating-comprehensive-llm-evaluation-framework-for-snap-benefits.

Zhou, Ruiwen, Wenyue Hua, Liangming Pan, et al. 2025. "RuleArena: A Benchmark for Rule-Guided Reasoning with LLMs in Real-World Scenarios." Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers) (Vienna, Austria), 550–72. https://doi.org/10.18653/v1/2025.acl-long.27.

Zhu, Fengbin, Wenqiang Lei, Youcheng Huang, et al. 2021. "TAT-QA: A Question Answering Benchmark on a Hybrid of Tabular and Textual Content in Finance." arXiv Preprint arXiv:2105.07624. https://arxiv.org/abs/2105.07624.